

基于 Gradio 的垃圾分类识别系统

实验文档（一）登录功能实现 & （二）模型训练功能实现

使用说明：本文档按「步骤 + 完整代码」编写。请严格按步骤顺序操作：**每完成一步先保存、再按文档中的「验证」命令检查，通过后再做下一步。**Web 界面采用 **Gradio**，不使用 Flask。

学习路径：

章节	目标	完成后应能做到
第 0 章	环境、数据库、初始用户	MySQL 可连、 <code>admin/123</code> 可查到
第（一）章	登录 / 登出	空输入拦截、正确账号登录、退出回登录页
第（二）章	模型训练 + 完整主界面	训练日志、用户管理、改密等

实验总览

项目	说明
实验名称	垃圾分类识别系统——登录功能与模型训练功能实现
目标文件	<code>gradio_app.py</code> 、 <code>train/train_resnet50.py</code>
依赖文件	<code>config.py</code> 、 <code>db.py</code> 、 <code>garbage.sql</code> （项目已提供）
建议学时	4 学时（登录约 1.5 学时，训练约 2.5 学时）

第 0 章 实验环境准备

0.1 安装依赖

在项目根目录执行：

```
pip install -r requirements.txt
```

`requirements.txt` 内容如下：

```
gradio
pillow
torch
torchvision
pandas
pymysql
cryptography
werkzeug
```

0.2 创建 MySQL 数据库

登录 MySQL 后执行：

```
CREATE DATABASE IF NOT EXISTS garbage DEFAULT CHARSET utf8mb4;
```

0.2.1 导入初始数据

在项目根目录执行（将 `garbage.sql` 导入数据库）：

```
mysql -u root -p garbage < garbage.sql
```

或在 Navicat / MySQL Workbench 中打开 `garbage.sql` 执行。

`users` 表中的密码字段为 **PBKDF2-SHA256 哈希**（非明文），默认测试账号如下：

用户名	密码	角色
admin	123 456	admin
123	123	普通用户

若库中仍是旧明文密码，登录会失败。可执行：

```
UPDATE users SET  
password='pbkdf2:sha256:1000000$BMgyvQe8BLIJIGdi$78e5792712a4807b3b62d4b328c7d7  
2f7951c2ddd10bd2574a9ea9befa4e8658' WHERE username IN ('admin','123456');  
(该哈希对应明文 123456)
```

0.3 配置 `config.py`

在项目根目录确认存在 `config.py`，内容如下（可按实际环境修改密码）：

```
import os  
from dataclasses import dataclass  
from urllib.parse import quote  
  
@dataclass(frozen=True)  
class AppConfig:  
    mysql_host: str = "127.0.0.1"  
    mysql_port: int = 3306  
    mysql_user: str = "root"  
    mysql_password: str = "root@123"  
    mysql_database: str = "garbage"  
    mysql_charset: str = "utf8mb4"  
  
    port: int = 6006  
  
    @classmethod  
    def load(cls) -> "AppConfig":  
        def _get_int(name: str, default: int) -> int:  
            v = (os.environ.get(name) or "").strip()  
            if not v:
```

```

        return default
    try:
        return int(v)
    except ValueError:
        return default

    return cls(
        mysql_host=(os.environ.get("MYSQL_HOST") or cls.mysql_host).strip(),
        mysql_port=_get_int("MYSQL_PORT", cls.mysql_port),
        mysql_user=(os.environ.get("MYSQL_USER") or cls.mysql_user).strip(),
        mysql_password=(os.environ.get("MYSQL_PASSWORD") or
cls.mysql_password),
        mysql_database=(os.environ.get("MYSQL_DATABASE") or
cls.mysql_database).strip(),
        mysql_charset=(os.environ.get("MYSQL_CHARSET") or
cls.mysql_charset).strip(),
        port=_get_int("PORT", cls.port),
    )

    def mysql_url(self) -> str:
        user = quote(self.mysql_user, safe='')
        password = quote(self.mysql_password, safe='')
        host = self.mysql_host
        port = int(self.mysql_port)
        db = quote(self.mysql_database, safe='')
        charset = quote(self.mysql_charset, safe='')
        return f"mysql+pymysql://{user}:{password}@{host}:{port}/{db}?charset=
{charset}"

    def db_url(self) -> str:
        env_db_url = (os.environ.get("DB_URL") or "").strip()
        return env_db_url or self.mysql_url()

```

0.4 确认项目目录

```

garbage-classifier-flask/
├─ config.py          ← 第 0 章创建/确认
├─ db.py              ← 项目自带（用户表、模型表等）
├─ garbage.sql        ← 初始数据（含哈希密码用户）
├─ gradio_app.py      ← 第 1、2 章逐步编写
├─ data/              ← 数据集目录
└─ train/             ← 第 2 章创建训练脚本

```

0.5 环境自检（进入第 1 章前必做）

在项目根目录依次执行，**全部通过**后再写登录代码：

```

pip install -r requirements.txt
python -c "from config import AppConfig; from db import Database;
db=Database(AppConfig.load().db_url()); db.init_tables(); print('MySQL 连接成功')"

```

在 MySQL 中确认已有测试用户（密码字段为 `pbkdf2:sha256:...` 开头）：

```
USE garbage;
SELECT id, username, LEFT(password, 20) AS pwd_prefix, role FROM users;
```

检查项	通过标准
依赖安装	无报错，且 <code>pip show werkzeug</code> 有输出
数据库连接	终端打印 MySQL 连接成功
初始用户	至少存在 <code>admin</code> 、 <code>123</code> ，且 <code>pwd_prefix</code> 为 <code>pbkdf2:sha256:100000</code>
配置一致	<code>config.py</code> 中主机、端口、密码与本机 MySQL 一致

常见失败：未导入 `garbage.sql` → `users` 表为空，登录永远失败；`config.py` 密码错误 → 连接失败。

（一）登录功能实现

一、实验目的

1. 使用 Gradio 搭建登录界面。
2. 使用 `gr.State` 保存登录态。
3. 使用 `.click().then()` 实现登录后切换面板。
4. 对接 MySQL `users` 表完成数据库登录校验。
5. 使用 Werkzeug **PBKDF2-SHA256** 对密码加密存储，登录时用哈希比对。

二、实现步骤总览

步骤	内容	验证方式
1.1	创建文件、导入模块、CSS	<code>python -m py_compile gradio_app.py</code>
1.2	数据库连接与密码工具函数	<code>python -c</code> 连库成功
1.3	登录/登出与密码管理函数	—
1.4	登录面板 UI	—
1.5	主面板（仅个人信息 + 退出）	—
1.6	<code>build_app</code> 与事件绑定	<code>python gradio_app.py</code> 可打开页面
1.7	本章功能测试（T1-T3、T6）	完成测试表
1.8	常见问题排查	遇错时查阅

第 1 章主面板为**精简版**（只有「个人信息」页签）。用户管理、改密界面在第（二）章扩展；密码创建/改密**函数**在本章先写好，便于第 2 章直接绑定按钮。

步骤 1.1 创建 `gradio_app.py` 并写入基础框架

操作：在项目根目录新建 `gradio_app.py`，粘贴以下代码：

```
"""Gradio 教学版 - 登录功能实验"""

from __future__ import annotations

import os
from typing import Any

import gradio as gr
from werkzeug.security import check_password_hash, generate_password_hash

from config import AppConfig
from db import Database

_CFG = AppConfig.load()
_DB: Database | None = None

APP_CSS = """
#login-wrap {
    max-width: 420px;
    margin: 10vh auto 0 auto;
    padding: 18px;
    border: 1px solid rgba(0,0,0,.08);
    border-radius: 12px;
    background: rgba(255,255,255,.92);
    box-shadow: 0 8px 30px rgba(0,0,0,.08);
}

#main-panel {
    max-width: 1180px;
    margin: 0 auto;
    padding: 14px 12px 24px 12px;
}

#main-panel::before {
    content: "";
    position: fixed;
    inset: 0;
    z-index: -1;
    background:
        radial-gradient(1200px 800px at 10% 10%, rgba(59,130,246,.08),
transparent 55%),
        radial-gradient(1200px 800px at 90% 0%, rgba(168,85,247,.08), transparent
55%),
        #f8fafc;
}
"""
```

步骤 1.2 添加数据库连接与密码工具函数

操作：在 `APP_CSS` 下方继续追加：

```
def _get_db() -> Database:
    global _DB
    if _DB is None:
        _DB = Database(_CFG.db_url())
        _DB.init_tables()
    return _DB

def _row_to_dict(row: Any) -> dict[str, Any]:
    if row is None:
        return {}
    if isinstance(row, dict):
        return dict(row)
    if hasattr(row, "keys"):
        return {key: row[key] for key in row.keys()}
    return {}

def _is_hashed_password(stored_password: str) -> bool:
    return str(stored_password or "").startswith(("pbkdf2:", "scrypt:",
"argon2:"))

def _verify_password(stored_password: str, provided_password: str) -> bool:
    """登录校验：仅对库中 PBKDF2 等哈希密文做 check_password_hash。"""
    stored = str(stored_password or "")
    provided = provided_password or ""
    if not stored or not provided or not _is_hashed_password(stored):
        return False
    try:
        return check_password_hash(stored, provided)
    except Exception:
        return False

def _hash_password(password: str) -> str:
    """保存密码前加密（PBKDF2-SHA256）。"""
    return generate_password_hash(password, method="pbkdf2:sha256")

def _is_admin(auth: Any) -> bool:
    return bool(auth and isinstance(auth, dict) and str(auth.get("role") or
"").lower() == "admin")

def _auth_user_id(auth: Any) -> int | None:
    if not auth or not isinstance(auth, dict):
        return None
    try:
        user_id = int(auth.get("user_id") or 0)
    except (TypeError, ValueError):
```

```
        return None
    return user_id if user_id > 0 else None
```

验证：确认能连上数据库（须已完成第 0.5 节导入 `garbage.sql`）：

```
python -c "from gradio_app import _get_db, _hash_password, _verify_password;
db=_get_db(); u=db.get_user_by_username('admin'); print('admin 存在:', bool(u));
h=_hash_password('test'); print('哈希前缀:', h[:30])"
```

预期：输出 `admin 存在: True` 和 `哈希前缀: pbkdf2:sha256:1000000$...`。

步骤 1.3 添加登录、登出与密码管理函数

操作：继续追加：

```
def _login(username: str, password: str):
    username = (username or "").strip()
    password = password or ""
    if not username:
        return None, "请输入用户名"
    if not password:
        return None, "请输入密码"

    try:
        row = _get_db().get_user_by_username(username)
    except Exception as exc:
        return None, f"登录失败: {exc}"

    user = _row_to_dict(row)
    if not user:
        return None, "用户名或密码错误"
    if not _verify_password(str(user.get("password") or ""), password):
        return None, "用户名或密码错误"

    is_active = user.get("is_active", 1)
    if is_active in (0, False, "0"):
        return None, "账号已被禁用, 请联系管理员"

    display_name = (user.get("name") or user.get("username") or username).strip()
    return {
        "user_id": user.get("id"),
        "username": user.get("username") or username,
        "role": user.get("role") or "user",
        "name": display_name,
    }, f"欢迎, {display_name}"

def _logout():
    return None, "已退出登录"

def _users_create(auth, username, password, role, name, gender, email, phone):
    if not _is_admin(auth):
```

```

        return "无权限：仅管理员可创建用户"
    username = (username or "").strip()
    password = password or ""
    if not username:
        return "请输入用户名"
    if not password:
        return "请输入密码"
    role = (role or "user").strip() or "user"
    try:
        db = _get_db()
        if db.get_user_by_username(username):
            return f"用户名已存在: {username}"
        db.create_user(
            username,
            _hash_password(password),
            role,
            (name or "").strip(),
            (gender or "").strip(),
            (email or "").strip(),
            (phone or "").strip(),
        )
        return f"用户创建成功: {username}"
    except Exception as exc:
        return f"创建失败: {exc}"

def _users_reset_password(auth, user_id, new_password):
    if not _is_admin(auth):
        return "无权限：仅管理员可重置密码"
    try:
        uid = int(user_id or 0)
    except (TypeError, ValueError):
        return "请输入有效的用户 ID"
    if uid <= 0:
        return "请输入有效的用户 ID"
    new_password = new_password or ""
    if not new_password:
        return "请输入新密码"
    try:
        db = _get_db()
        if not _row_to_dict(db.get_user_by_id(uid)):
            return f"用户不存在: {uid}"
        db.update_user_password(uid, _hash_password(new_password))
        return f"用户 {uid} 密码已重置"
    except Exception as exc:
        return f"重置失败: {exc}"

def _profile_change_password(auth, old_pwd, new_pwd):
    user_id = _auth_user_id(auth)
    if not user_id:
        return "请先登录"
    old_pwd = old_pwd or ""
    new_pwd = (new_pwd or "").strip()
    if not old_pwd:
        return "请输入旧密码"

```



```

if not new_pwd:
    return "请输入新密码"
try:
    user = _row_to_dict(_get_db().get_user_by_id(user_id))
    if not user:
        return "用户不存在"
    if not _verify_password(str(user.get("password") or ""), old_pwd):
        return "旧密码错误"
    _get_db().update_user_password(user_id, _hash_password(new_pwd))
    return "密码修改成功"
except Exception as exc:
    return f"密码修改失败: {exc}"

def _profile_get(auth):
    return {
        "name": "示例用户",
        "gender": "未知",
        "email": "demo@example.com",
        "phone": "13000000000",
    }, "个人信息加载完成"

```

说明:

- `db.create_user` / `db.update_user_password` 接收的 `password` 必须是已哈希的字符串, 由 `_hash_password()` 生成。
- 登录时**不会**把用户输入与明文密码直接比较, 只通过 `_verify_password` → `check_password_hash` 校验。

步骤 1.4 添加登录面板 UI

操作: 继续追加:

```

def build_login_panel() -> dict[str, Any]:
    """构建登录表单。"""
    panel = gr.Column(visible=True, elem_id="login-panel")
    with panel:
        with gr.Column(elem_id="login-wrap"):
            gr.Markdown("## 登录")
            username = gr.Textbox(label="用户名")
            password = gr.Textbox(label="密码", type="password")
            login_btn = gr.Button("登录", variant="primary")
            status = gr.Textbox(label="状态", interactive=False)

    return {
        "panel": panel,
        "username": username,
        "password": password,
        "login_btn": login_btn,
        "status": status,
    }

```

步骤 1.5 添加主面板（登录成功后显示）

操作：继续追加：

```
def build_main_panel(auth_state) -> dict[str, Any]:
    """构建主工作区（第 1 章精简版）。登录前隐藏，登录后显示。"""
    panel = gr.Column(visible=False, elem_id="main-panel")

    with panel:
        gr.Markdown("## 欢迎使用垃圾分类识别系统")
        with gr.Tabs():
            with gr.Tab("个人信息"):
                logout_btn = gr.Button("退出登录")
                pf_name = gr.Textbox(label="姓名")
                pf_gender = gr.Textbox(label="性别")
                pf_email = gr.Textbox(label="邮箱")
                pf_phone = gr.Textbox(label="电话")

    return {
        "panel": panel,
        "logout_btn": logout_btn,
        "profile_fill_outputs": [pf_name, pf_gender, pf_email, pf_phone],
    }
```

说明：第（二）章会把 `build_main_panel` 扩展为含「图像识别、识别记录、模型管理、用户管理」等页签的完整版，并传入 `callbacks` 字典。**不要删除**本章已写的 `_login`、`_users_create` 等函数，第 2 章在其基础上追加即可。

步骤 1.6 添加 `build_app` 并绑定登录/登出事件

操作：继续追加以下完整代码：

```
def build_app() -> gr.Blocks:
    with gr.Blocks(title="垃圾分类识别系统", css=APP_CSS) as demo:
        auth_state = gr.State(None)

        gr.Markdown("## 垃圾分类识别系统（教学页面）")

        def _apply_auth_ui(auth, status: str):
            """根据是否登录，切换登录面板与主面板。"""
            logged_in = bool(auth and isinstance(auth, dict) and
                              auth.get("user_id"))
            return (
                gr.update(visible=not logged_in),
                gr.update(visible=logged_in),
                "" if logged_in else ((status or "").strip() or "请登录后继续"),
            )

        def _pf_fill(auth):
            if not auth:
                return "", "", "", ""
            data, _ = _profile_get(auth)
            return (
```

```

        data.get("name", ""),
        data.get("gender", ""),
        data.get("email", ""),
        data.get("phone", ""),
    )

    login_ui = build_login_panel()
    main_ui = build_main_panel(auth_state)

    # 启动时短暂显示主界面再隐藏, 避免 Gradio 首次登录后主面板空白。
    demo.load(lambda: gr.update(visible=True), inputs=None, outputs=
[main_ui["panel"]]).then(
        lambda: gr.update(visible=False),
        inputs=None,
        outputs=[main_ui["panel"]],
    )

    # ----- 登录事件链 -----
    login_ui["login_btn"].click(
        _login,
        inputs=[login_ui["username"], login_ui["password"]],
        outputs=[auth_state, login_ui["status"]],
    ).then(
        _apply_auth_ui,
        inputs=[auth_state, login_ui["status"]],
        outputs=[login_ui["panel"], main_ui["panel"], login_ui["status"]],
    ).then(
        _pf_fill,
        inputs=[auth_state],
        outputs=main_ui["profile_fill_outputs"],
    )

    # ----- 登出事件链 -----
    main_ui["logout_btn"].click(
        _logout,
        inputs=None,
        outputs=[auth_state, login_ui["status"]],
    ).then(
        _apply_auth_ui,
        inputs=[auth_state, login_ui["status"]],
        outputs=[login_ui["panel"], main_ui["panel"], login_ui["status"]],
    ).then(
        lambda: ("", "", "", ""),
        inputs=None,
        outputs=main_ui["profile_fill_outputs"],
    )

    return demo

if __name__ == "__main__":
    app = build_app()
    app.launch(
        server_name="0.0.0.0",
        server_port=int(os.environ.get("PORT", _CFG.port)),
    )

```

步骤 1.7 运行并验证登录功能（第 1 章测试）

前置：已执行第 0.5 节自检，且已导入 `garbage.sql`。未导入则无法通过 T3。

操作：

```
python gradio_app.py
```

浏览器打开 `http://localhost:6006`（若设置了 `PORT` 环境变量则使用对应端口）。

第（一）章必测用例

编号	操作	预期结果
T1	用户名或密码留空，点击登录	状态栏提示「请输入用户名」或「请输入密码」，不进入主界面
T2	错误用户名或密码	提示「用户名或密码错误」
T3	<code>admin / 123</code> 登录	进入主界面，可见「欢迎使用...」和「个人信息」页签
T6	点击「退出登录」	回到登录页，主面板隐藏

第（二）章完成后补测（密码管理 UI）

编号	操作	预期结果
T4	管理员在「用户管理」创建用户	提示创建成功；库中 <code>password</code> 为 <code>pbkdf2:sha256:...</code>
T5	用新账号登录	登录成功
T7	个人信息页「修改密码」	旧密码错误时提示「旧密码错误」；正确则改密成功

步骤 1.8 常见问题排查

现象	可能原因	处理办法
启动报错 <code>No module named 'werkzeug'</code>	未安装依赖	<code>pip install werkzeug</code> 或 <code>pip install -r requirements.txt</code>
登录提示「登录失败：...」含数据库错误	MySQL 未启动或 <code>config.py</code> 配置错误	检查服务与账号密码，重做 0.5 自检
始终「用户名或密码错误」	未导入 <code>garbage.sql</code> 或库中仍是明文密码	导入 SQL 或执行 0.2.1 中的 <code>UPDATE</code> 语句
登录成功但主界面空白	未写 <code>demo.load</code> 预渲染	对照步骤 1.6 补全两段 <code>demo.load</code>

现象	可能原因	解决办法
页面顶部出现「登录 / 系统」小页签	误用了 Tabs 切换方案	按本文档使用「双 Column + visible」方案

附录 1-A 登录与密码安全要点（自查清单）

要点	实现位置
密码哈希算法	Werkzeug <code>pbkdf2:sha256</code> (<code>_hash_password</code>)
登录校验	<code>_verify_password</code> → <code>check_password_hash</code>
创建用户	<code>_users_create</code> 入库前 <code>_hash_password</code>
管理员重置密码	<code>_users_reset_password</code>
用户自助改密	<code>_profile_change_password</code> (先验旧密码)
库中禁止存明文	<code>db.create_user</code> / <code>update_user_password</code> 注释已说明

完整可运行代码以项目根目录 `gradio_app.py` 为准（含登录、识别、记录、模型训练等全部页签）。若中间步骤出错，可先对照仓库中的 `gradio_app.py` 第 1 章相关函数，再进入第（二）章追加训练代码。

附录 1-B 第 1 章完成标准（教师/学生对照）

完成第（一）章后，`gradio_app.py` 中至少应包含：

- ☐ `_get_db`、`_row_to_dict`、`_hash_password`、`_verify_password`
- ☐ `_login`、`_logout`（数据库校验 + 哈希比对）
- ☐ `_users_create`、`_users_reset_password`、`_profile_change_password`（可先不绑 UI）
- ☐ `build_login_panel`、`build_main_panel`（精简版）、`build_app`（含 `demo.load`）
- ☐ 测试 T1、T2、T3、T6 全部通过

（二）模型训练功能实现

前置条件：第（一）章 T1-T3、T6 已通过。
重要：本章在已有 `gradio_app.py` 上追加/替换局部代码，不要整文件覆盖，以免丢失第 1 章的登录逻辑。

一、实验目的

- 在 Gradio 页面中调度外部 Python 训练脚本。
- 使用 `subprocess.Popen` 实时显示训练日志与进度。
- 训练完成后自动将 `.pth` 权重登记到 MySQL。

二、实现步骤总览

步骤	内容	验证方式
2.1	补充 import 与全局变量	—
2.2	添加数据库与模型登记函数	可连 MySQL
2.3	添加训练调度函数 <code>_train_model</code>	—
2.4	扩展主面板，加入「模型管理」页签	界面可见训练面板
2.5	创建 <code>train/train_resnet50.py</code>	命令行可训练
2.6	扩展 <code>build_main_panel</code> 、更新 <code>build_app</code>	页面可启动训练
2.7	功能测试（M1-M4、T4-T5、T7）	完成测试表

步骤 2.1 补充 import 与全局变量

操作：在保留第 1 章全部登录相关函数的前提下，将文件头部合并为（不要删除 `werkzeug`、`Database`、`_login` 等已有内容）：

```
"""Gradio 教学版 - 登录 + 模型训练"""

from __future__ import annotations

import locale
import os
import re
import subprocess
import sys
from datetime import datetime
from typing import Any

import gradio as gr
import torch
from werkzeug.security import check_password_hash, generate_password_hash

from config import AppConfig
from db import Database
```

在 `_CFG = AppConfig.load()` 之后追加：

```
_EPOCH_PROGRESS_RE = re.compile(r"Epoch\s+(\d+)\s*/\s*(\d+)\s*" | Epoch\s+(\d+)/(\d+)")
_TRAIN_NUM_WORKERS = 2
_TRAIN_FORCE_CPU = False
_TRAIN_RESNET50_PRETRAINED = True
_CURRENT_MODEL_KEY = "current_model"
_DB: Database | None = None
```

追加路径工具函数：

```

def _project_root() -> str:
    return os.path.dirname(os.path.abspath(__file__))

def _model_output_dir(model_name: str) -> str:
    return os.path.join(_project_root(), "models", model_name)

def _to_relative_project_path(path: str) -> str:
    abs_path = os.path.abspath(path)
    root = _project_root()
    try:
        rel_path = os.path.relpath(abs_path, root)
    except ValueError:
        return abs_path
    return rel_path.replace("/", os.sep)

def _items_to_rows(items: list[dict[str, Any]], fields: list[str]) -> list[list[Any]]:
    return [[item.get(field) for field in fields] for item in (items or [])]

def _models_to_rows(items: list[dict[str, Any]]) -> list[list[Any]]:
    return _items_to_rows(items, ["name", "size_mb", "num_classes", "epoch",
    "val_acc", "updated_at", "is_current"])

```

步骤 2.2 添加数据库与模型登记函数

操作：在路径工具函数下方追加：

```

def _get_db() -> Database:
    global _DB
    if _DB is None:
        _DB = Database(_CFG.db_url())
        _DB.init_tables()
    return _DB

def _row_to_dict(row: Any) -> dict[str, Any]:
    if row is None:
        return {}
    if isinstance(row, dict):
        return dict(row)
    if hasattr(row, "keys"):
        return {key: row[key] for key in row.keys()}
    return {}

def _list_models_from_db() -> list[dict[str, Any]]:
    db = _get_db()
    current_name = (db.get_setting(_CURRENT_MODEL_KEY, "") or "").strip()
    items: list[dict[str, Any]] = []
    for row in db.list_models():

```

```

        item = _row_to_dict(row)
        item["is_current"] = item.get("name") == current_name
        items.append(item)
    return items

def _get_current_model_name_from_db() -> str | None:
    db = _get_db()
    current_name = (db.get_setting(_CURRENT_MODEL_KEY, "") or "").strip()
    items = _list_models_from_db()
    valid_names = {str(item.get("name") or "").strip() for item in items if
item.get("name")}
    if current_name and current_name in valid_names:
        return current_name
    if current_name:
        db.set_setting(_CURRENT_MODEL_KEY, "")
    if not items:
        return None
    return items[0].get("name")

def _register_model_record(checkpoint_path: str, fallback_arch: str) -> None:
    db = _get_db()
    checkpoint_path = os.path.abspath(checkpoint_path)
    if not os.path.exists(checkpoint_path):
        return

    checkpoint = torch.load(checkpoint_path, map_location="cpu")
    arch = str(checkpoint.get("arch") or fallback_arch or "")
    class_names = checkpoint.get("class_names") or []
    num_classes = checkpoint.get("num_classes")
    if num_classes is None and isinstance(class_names, list):
        num_classes = len(class_names)

    epoch = checkpoint.get("epoch")
    val_acc = checkpoint.get("val_acc")
    size_mb = round(os.path.getsize(checkpoint_path) / (1024 * 1024), 2)
    now_text = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    name = os.path.basename(checkpoint_path)
    relative_path = _to_relative_project_path(checkpoint_path)

    if db.check_model_exists(name):
        db.update_model(name, relative_path, arch, size_mb, num_classes, epoch,
val_acc, now_text)
    else:
        db.create_model(name, relative_path, arch, size_mb, num_classes, epoch,
val_acc, now_text, now_text)

def _refresh_weights():
    try:
        return [item["name"] for item in _list_models_from_db()]
    except Exception:
        return []

```



```

def _get_current_weight():
    try:
        return _get_current_model_name_from_db()
    except Exception:
        weights = _refresh_weights()
        return weights[0] if weights else None

def _models_list(auth):
    try:
        items = _list_models_from_db()
        current = _get_current_model_name_from_db()
        return items, current or "", f"共 {len(items)} 个模型"
    except Exception as exc:
        return [], None, f"加载模型列表失败: {exc}"

def _model_panel_state(items, cur, msg, choices):
    return _models_to_rows(items), cur or "", msg, gr.update(choices=choices,
value=cur or None)

```

验证：在项目根目录执行：

```
python -c "from gradio_app import _get_db; _get_db(); print('数据库连接成功')"
```

步骤 2.3 添加训练调度函数（完整代码）

操作：继续追加以下三个函数：

```

def _parse_training_progress(line: str, fallback_total: int) -> tuple[int, int]:
    match = _EPOCH_PROGRESS_RE.search(line or "")
    if not match:
        return 0, fallback_total
    current = match.group(1) or match.group(3) or "0"
    total = match.group(2) or match.group(4) or str(fallback_total)
    try:
        return max(int(current), 0), max(int(total), 1)
    except ValueError:
        return 0, fallback_total

def _sanitize_training_line(line: str) -> str:
    text = (line or "").replace("\r", "").rstrip()
    if not text:
        return ""
    text = text.replace("\ufffd", "")
    text = re.sub(r"\?{3,}", "", text)
    if "%" in text and "|" in text and "/" in text:
        text = re.sub(r"[\x20-\x7E\u4e00-\u9fff]+", "", text)
        text = text.replace("||", "|")
    return text.strip()

```

```

def _train_model(auth, model_name, save_path, epochs, batch_size, lr,
weight_decay):
    """Gradio 生成器: 启动子进程训练并流式返回日志。"""
    model_name = (model_name or "").strip().lower()
    save_path = (save_path or "").strip() or _model_output_dir(model_name)
    total_epochs = max(int(epochs or 1), 1)

    script_map = {
        "resnet50": "train_resnet50.py",
        "alexnet": "train_alexnet.py",
        "cnn": "train_cnn.py",
    }
    script_name = script_map.get(model_name)
    if not script_name:
        yield "请选择要训练的模型。", "未开始", 0
        return

    train_dir = os.path.join(_project_root(), "train")
    script_path = os.path.join(train_dir, script_name)
    if not os.path.exists(script_path):
        yield f"未找到训练脚本: {script_path}", "训练脚本不存在", 0
        return

    os.makedirs(save_path, exist_ok=True)
    run_name = f"{model_name}_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

    cmd = [
        sys.executable,
        "-u",
        script_path,
        "--out-dir",
        save_path,
        "--epochs",
        str(total_epochs),
        "--batch-size",
        str(int(batch_size or 8)),
        "--lr",
        str(float(lr or 1e-3)),
        "--weight-decay",
        str(float(weight_decay or 1e-4)),
        "--num-workers",
        str(_TRAIN_NUM_WORKERS),
        "--run-name",
        run_name,
    ]

    if _TRAIN_FORCE_CPU:
        cmd.append("--cpu")

    if model_name == "resnet50" and _TRAIN_RESNET50_PRETRAINED:
        cmd.append("--pretrained")

    yield "开始训练...\n" + " ".join(cmd), "正在启动训练进程", 0

    child_env = os.environ.copy()
    child_env["PYTHONIOENCODING"] = "utf-8"

```

```

child_env["PYTHONUTF8"] = "1"
child_env["TQDM_ASCII"] = "1"

process = subprocess.Popen(
    cmd,
    cwd=_project_root(),
    env=child_env,
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    text=True,
    encoding=locale.getpreferredencoding(False) or "utf-8",
    errors="replace",
    bufsize=1,
)

logs: list[str] = []
progress_value = 0
status_text = "训练中..."
saved_checkpoint_path = ""

assert process.stdout is not None
for line in process.stdout:
    clean_line = _sanitize_training_line(line)
    if not clean_line:
        continue

    logs.append(clean_line)
    if clean_line.startswith("saved best checkpoint to:"):
        saved_checkpoint_path = clean_line.split(":", 1)[1].strip()

    current_epoch, parsed_total = _parse_training_progress(clean_line,
total_epochs)
    if current_epoch > 0:
        progress_value = min(int(current_epoch * 100 / max(parsed_total, 1)),
100)

        status_text = f"训练中: 第 {current_epoch}/{parsed_total} 轮"
    else:
        status_text = clean_line

    yield "\n".join(logs[-400:]), status_text, progress_value

return_code = process.wait()
if return_code == 0:
    if not saved_checkpoint_path:
        saved_checkpoint_path = os.path.join(save_path, f"{run_name}.pth")
    try:
        _register_model_record(saved_checkpoint_path, model_name)
        _get_db().set_setting(_CURRENT_MODEL_KEY,
os.path.basename(saved_checkpoint_path))
    except Exception as exc:
        logs.append(f"[数据库写入失败] {exc}")
        logs.append("[训练完成]")
        status_text = "训练完成"
        progress_value = 100
    else:
        logs.append(f"[训练失败] 退出码: {return_code}")

```

```
status_text = f"训练失败（退出码 {return_code}）"
```

```
yield "\n".join(logs[-400:]), status_text, progress_value
```

步骤 2.4 扩展主面板——添加「模型管理」页签

操作：将 `build_main_panel` 函数整体替换为以下完整代码：

```
def build_main_panel(auth_state, callbacks: dict[str, Any]) -> dict[str, Any]:
    panel = gr.Column(visible=False, elem_id="main-panel")

    with panel:
        gr.Markdown("## 欢迎使用垃圾分类识别系统")
        with gr.Tabs():
            with gr.Tab("个人信息"):
                logout_btn = gr.Button("退出登录")
                pf_name = gr.Textbox(label="姓名")
                pf_gender = gr.Textbox(label="性别")
                pf_email = gr.Textbox(label="邮箱")
                pf_phone = gr.Textbox(label="电话")

            with gr.Tab("模型管理（管理员）"):
                models_refresh = gr.Button("刷新模型列表")
                models_table = gr.DataFrame(
                    label="模型列表",
                    headers=["name", "size_mb", "num_classes", "epoch",
                             "val_acc", "updated_at", "is_current"],
                    interactive=False,
                )
                current_model = gr.Textbox(label="当前模型", interactive=False)
                models_msg = gr.Textbox(label="提示", interactive=False)
                set_current_dd = gr.Dropdown(
                    label="设置当前模型",
                    choices=callbacks["refresh_weights"](),
                )
                set_current_btn = gr.Button("设置")
                set_current_msg = gr.Textbox(label="设置提示", interactive=False)

                gr.Markdown("### 模型训练")
                with gr.Accordion("训练模型面板", open=True):
                    train_model_name = gr.Dropdown(
                        label="选择模型",
                        choices=["resnet50", "alexnet", "cnn"],
                        value="resnet50",
                    )
                    train_save_path = gr.Textbox(
                        label="模型保存目录",
                        value=_model_output_dir("resnet50"),
                        interactive=True,
                    )
                    with gr.Row():
                        train_epochs = gr.Number(label="训练轮数", value=2,
precision=0)
```

```

        train_batch_size = gr.Number(label="批大小", value=16,
precision=0)

        with gr.Row():
            train_lr = gr.Number(label="学习率", value=0.001,
precision=6)

            train_weight_decay = gr.Number(label="权重衰减",
value=0.0001, precision=6)

            train_btn = gr.Button("开始训练", variant="primary")
            train_status = gr.Textbox(label="训练状态", value="未开始",
interactive=False)

            train_progress = gr.Slider(
                label="训练进度", minimum=0, maximum=100, step=1, value=0,
interactive=False
            )

            train_log = gr.Textbox(label="训练日志", lines=18,
max_lines=24, interactive=False)

    def _models_refresh(auth):
        items, cur, msg = callbacks["models_list"](auth)
        choices = callbacks["refresh_weights"]()
        return _model_panel_state(items, cur, msg, choices)

    def _update_train_save_path(model_name):
        model_name = (model_name or "resnet50").strip().lower()
        return _model_output_dir(model_name)

    models_refresh.click(
        _models_refresh,
        inputs=[auth_state],
        outputs=[models_table, current_model, models_msg,
set_current_dd],
    )

    set_current_btn.click(
        lambda auth, name: f"已选择模型: {name}",
        inputs=[auth_state, set_current_dd],
        outputs=[set_current_msg],
    )

    train_model_name.change(
        _update_train_save_path,
        inputs=[train_model_name],
        outputs=[train_save_path],
    )

    train_btn.click(
        callbacks["train_model"],
        inputs=[
            auth_state,
            train_model_name,
            train_save_path,
            train_epochs,
            train_batch_size,
            train_lr,
            train_weight_decay,
        ],
        outputs=[train_log, train_status, train_progress],
    ).then(
        _models_refresh,

```

```

        inputs=[auth_state],
        outputs=[models_table, current_model, models_msg,
        set_current_dd],
    )

    return {
        "panel": panel,
        "logout_btn": logout_btn,
        "profile_fill_outputs": [pf_name, pf_gender, pf_email, pf_phone],
        "models_outputs": [models_table, current_model, models_msg,
        set_current_dd],
    }

```

步骤 2.5 创建训练脚本 train/train_resnet50.py

操作：新建目录 `train/`，创建文件 `train/train_resnet50.py`，粘贴以下完整代码：

```

import argparse
import os
from dataclasses import dataclass
from datetime import datetime

import torch
import torch.nn as nn
import torch.optim as optim
from PIL import Image
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms

@dataclass(frozen=True)
class Sample:
    path: str
    label: int

class GarbageDataset(Dataset):
    def __init__(self, root_dir: str, list_file: str, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.samples: list[Sample] = []

        with open(list_file, "r", encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                parts = line.split()
                if len(parts) != 2:
                    continue
                rel_path, label = parts
                abs_path = os.path.join(root_dir, rel_path.replace("/", os.sep))
                self.samples.append(Sample(path=abs_path, label=int(label)))

```

```

def __len__(self):
    return len(self.samples)

def __getitem__(self, idx: int):
    s = self.samples[idx]
    img = Image.open(s.path).convert("RGB")
    if self.transform is not None:
        img = self.transform(img)
    return img, s.label

def build_transforms(train: bool):
    if train:
        return transforms.Compose(
            [
                transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2),
                transforms.ToTensor(),
                transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])),
            ]
        )
    return transforms.Compose(
        [
            transforms.Resize(256),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])),
        ]
    )

def load_labels(label_list_path: str):
    labels: list[str] = []
    with open(label_list_path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                labels.append(line)
    return labels

def evaluate(model, loader, device):
    model.eval()
    correct = 0
    total = 0
    loss_sum = 0.0
    criterion = nn.CrossEntropyLoss()

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)

```

```

        logits = model(x)
        loss = criterion(logits, y)
        loss_sum += float(loss.item()) * x.size(0)
        pred = torch.argmax(logits, dim=1)
        correct += int((pred == y).sum().item())
        total += int(y.numel())

    avg_loss = loss_sum / max(total, 1)
    acc = correct / max(total, 1)
    return avg_loss, acc

def train(args):
    torch.manual_seed(args.seed)

    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    args.data_dir = os.path.join(base_dir, args.data_dir) if not
os.path.isabs(args.data_dir) else args.data_dir
    args.train_list = os.path.join(base_dir, args.train_list) if not
os.path.isabs(args.train_list) else args.train_list
    args.val_list = os.path.join(base_dir, args.val_list) if not
os.path.isabs(args.val_list) else args.val_list
    args.label_list = os.path.join(base_dir, args.label_list) if not
os.path.isabs(args.label_list) else args.label_list
    args.out_dir = os.path.join(base_dir, args.out_dir) if not
os.path.isabs(args.out_dir) else args.out_dir

    device = torch.device("cuda" if torch.cuda.is_available() and not args.cpu
else "cpu")

    train_ds = GarbageDataset(args.data_dir, args.train_list,
transform=build_transforms(train=True))
    val_ds = GarbageDataset(args.data_dir, args.val_list,
transform=build_transforms(train=False))

    train_loader = DataLoader(
        train_ds,
        batch_size=args.batch_size,
        shuffle=True,
        num_workers=args.num_workers,
        pin_memory=(device.type == "cuda"),
    )
    val_loader = DataLoader(
        val_ds,
        batch_size=args.batch_size,
        shuffle=False,
        num_workers=args.num_workers,
        pin_memory=(device.type == "cuda"),
    )

    class_names = load_labels(args.label_list)
    num_classes = len(class_names)

    model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2 if
args.pretrained else None)
    model.fc = nn.Linear(model.fc.in_features, num_classes)

```



```

model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=args.lr,
weight_decay=args.weight_decay)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=args.epochs)

best_acc = -1.0
os.makedirs(args.out_dir, exist_ok=True)
run_name = args.run_name or
f"resnet50_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
out_path = os.path.join(args.out_dir, f"{run_name}.pth")

for epoch in range(1, args.epochs + 1):
    model.train()
    running = 0.0
    seen = 0

    for x, y in train_loader:
        x = x.to(device)
        y = y.to(device)
        optimizer.zero_grad(set_to_none=True)
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
        running += float(loss.item()) * x.size(0)
        seen += x.size(0)

    scheduler.step()
    train_loss = running / max(seen, 1)
    val_loss, val_acc = evaluate(model, val_loader, device)

    if val_acc > best_acc:
        best_acc = val_acc
        ckpt = {
            "arch": "resnet50",
            "num_classes": num_classes,
            "class_names": class_names,
            "state_dict": model.state_dict(),
            "val_acc": best_acc,
            "epoch": epoch,
        }
        torch.save(ckpt, out_path)

    print(
        f"Epoch {epoch:03d}/{args.epochs} | "
        f"train_loss={train_loss:.4f} | val_loss={val_loss:.4f} | val_acc=
{val_acc:.4f} | best={best_acc:.4f}"
    )

    print(f"Saved best checkpoint to: {out_path}")

def parse_args():

```

```

p = argparse.ArgumentParser()
p.add_argument("--data-dir", default="data")
p.add_argument("--train-list", default="data/train_list.txt")
p.add_argument("--val-list", default="data/validate_list.txt")
p.add_argument("--label-list", default="data/label_list.txt")
p.add_argument("--out-dir", default="models")
p.add_argument("--run-name", default="")
p.add_argument("--epochs", type=int, default=10)
p.add_argument("--batch-size", type=int, default=32)
p.add_argument("--lr", type=float, default=3e-4)
p.add_argument("--weight-decay", type=float, default=1e-4)
p.add_argument("--num-workers", type=int, default=2)
p.add_argument("--seed", type=int, default=42)
p.add_argument("--cpu", action="store_true")
p.add_argument("--pretrained", action="store_true")
return p.parse_args()

if __name__ == "__main__":
    train(parse_args())

```

验证（命令行独立训练）：

```

python train/train_resnet50.py --epochs 1 --batch-size 8 --pretrained --out-dir
models/resnet50 --cpu

```

看到 Epoch 001/001 和 Saved best checkpoint to: 即表示脚本正常。

步骤 2.6 更新 build_app 注册训练回调

操作：将 build_app 函数整体替换为：

```

def build_app() -> gr.Blocks:
    with gr.Blocks(title="垃圾分类识别系统", css=APP_CSS) as demo:
        auth_state = gr.State(None)
        gr.Markdown("# 垃圾分类识别系统（教学页面）")

        def _apply_auth_ui(auth, status: str):
            logged_in = bool(auth and isinstance(auth, dict) and
auth.get("user_id"))
            return (
                gr.update(visible=not logged_in),
                gr.update(visible=logged_in),
                "" if logged_in else ((status or "").strip() or "请登录后续"),
            )

        def _pf_fill(auth):
            if not auth:
                return "", "", "", ""
            data, _ = _profile_get(auth)
            return data.get("name", ""), data.get("gender", ""),
data.get("email", ""), data.get("phone", "")

```

```

def _load_models_ui(auth):
    if not auth:
        return [], "", "请先登录", gr.update()
    try:
        items, cur, msg = _models_list(auth)
        return _model_panel_state(items, cur, msg, _refresh_weights())
    except Exception as exc:
        return [], "", f"加载模型列表失败: {exc}", gr.update()

login_ui = build_login_panel()
main_ui = build_main_panel(
    auth_state,
    {
        "get_current_weight": _get_current_weight,
        "models_list": _models_list,
        "models_set_current": _models_set_current,
        "train_model": _train_model,
        "models_upload": _models_upload,
        "profile_change_password": _profile_change_password,
        "profile_fill": _pf_fill,
        "profile_update": _profile_update,
        "record_detail": _record_detail,
        "records_export": _records_export,
        "records_query": _records_query,
        "recognize": _recognize,
        "refresh_weights": _refresh_weights,
        "users_create": _users_create,
        "users_delete": _users_delete,
        "users_list": _users_list,
        "users_reset_password": _users_reset_password,
        "users_update": _users_update,
    },
)

demo.load(lambda: gr.update(visible=True), inputs=None, outputs=
[main_ui["panel"]]).then(
    lambda: gr.update(visible=False),
    inputs=None,
    outputs=[main_ui["panel"]],
)

login_ui["login_btn"].click(
    _login,
    inputs=[login_ui["username"], login_ui["password"]],
    outputs=[auth_state, login_ui["status"]],
).then(
    _apply_auth_ui,
    inputs=[auth_state, login_ui["status"]],
    outputs=[login_ui["panel"], main_ui["panel"], login_ui["status"]],
).then(
    _pf_fill,
    inputs=[auth_state],
    outputs=main_ui["profile_fill_outputs"],
).then(
    _load_models_ui,
    inputs=[auth_state],

```

```

        outputs=main_ui["models_outputs"],
    ).then(
        _build_records_panel_state,
        inputs=[auth_state],
        outputs=main_ui["records_outputs"],
    )

    main_ui["logout_btn"].click(_logout, inputs=None, outputs=[auth_state,
login_ui["status"]]).then(
        _apply_auth_ui,
        inputs=[auth_state, login_ui["status"]],
        outputs=[login_ui["panel"], main_ui["panel"], login_ui["status"]],
    ).then(
        lambda: ("", "", "", ""),
        inputs=None,
        outputs=main_ui["profile_fill_outputs"],
    )

    return demo

if __name__ == "__main__":
    app = build_app()
    app.launch(server_name="0.0.0.0", server_port=int(os.environ.get("PORT",
_CFG.port)))

```

步骤 2.7 运行并验证模型训练功能

操作：

```
python gradio_app.py
```

1. 使用 `admin` / `123` 登录（需已导入 `garbage.sql` 且密码为 PBKDF2 哈希）。
2. 进入「模型管理（管理员）」页签。
3. 选择 `resnet50`，训练轮数 `2`，批大小 `16`。
4. 点击「开始训练」，观察日志与进度条。
5. 训练完成后点击「刷新模型列表」，确认出现新 `.pth` 记录。
6. （可选）在「用户管理」创建测试用户，完成 **T4-T5**；在「个人信息」测试 **T7** 改密。

编号	操作	预期结果
M1	页面启动训练 <code>resnet50</code> , <code>epochs=2</code>	日志逐行输出，进度条到 100%
M2	训练完成	显示 [训练完成]
M3	刷新模型列表	表格出现新模型及 <code>val_acc</code>
M4	检查 <code>models</code> 目录	存在 <code>.pth</code> 文件
T4	管理员创建用户	见第 1 章测试表

编号	操作	预期结果
T5	新用户登录	见第 1 章测试表
T7	修改密码	见第 1 章测试表

MySQL 验证：

```
SELECT name, arch, val_acc, epoch FROM models ORDER BY id DESC LIMIT 3;
```

附录 2-A 模型训练相关完整代码清单

完成第（二）章后，`gradio_app.py` 中应包含以下函数（便于自查）：

函数名	作用
<code>_hash_password</code> / <code>_verify_password</code>	密码哈希与登录校验
<code>_login</code> / <code>_logout</code>	数据库登录与登出
<code>_users_create</code> / <code>_users_reset_password</code>	创建用户、重置密码（哈希入库）
<code>_profile_change_password</code>	用户自助改密
<code>_get_db</code>	连接 MySQL 并建表
<code>_register_model_record</code>	训练产物入库
<code>_models_list</code>	查询模型列表
<code>_model_panel_state</code>	整理模型页输出
<code>_parse_training_progress</code>	解析 Epoch 进度
<code>_sanitize_training_line</code>	清洗训练日志
<code>_train_model</code>	子进程调度训练
<code>build_main_panel</code>	含模型管理页签
<code>build_app</code>	注册 <code>train_model</code> 回调

实验报告提交要求

- 登录功能截图 3 张（登录前 / 登录后 / 退出后）。
- 模型训练日志截图 1 张、模型列表截图 1 张。
- 填写测试用例表：**第 1 章提交 T1-T3、T6**；全部完成后补 **T4-T5、T7、M1-M4**。
- 回答思考题（至少 2 题）：
 - 为什么密码要哈希后再存入数据库，而不是保存明文？
 - `check_password_hash` 与简单 `==` 比较明文密码有何区别？
 - 为什么训练脚本通过 `subprocess` 调用，而不是写在 Gradio 回调里？

- `_train_model` 为什么使用 `yield` 而不是 `return`?
-

文档版本: v3.1 (优化教学路径: 环境自检、分章测试、常见问题)

更新日期: 2026-05-28